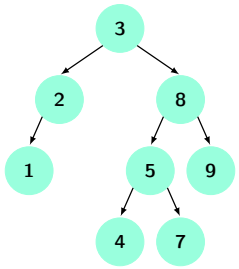
Week 10 Applied Sheet

(Solutions)

Useful advice: The following solutions pertain to the theoretical problems given in the applied classes. You are strongly advised to attempt the problems thoroughly before looking at these solutions. Simply reading the solutions without thinking about the problems will rob you of the practice required to be able to solve complicated problems on your own. You will perform poorly on the exam if you simply attempt to memorise solutions to these problems. Thinking about a problem, even if you do not solve it, will greatly increase your understanding of the underlying concepts. Solutions are typically not provided for Python implementation questions. In some cases, pseudocode may be provided where it illustrates a particular useful concept.

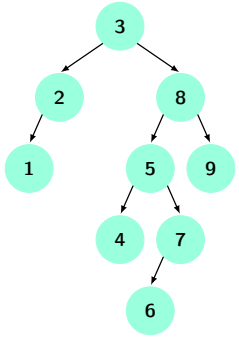
Problems

Problem 1. Insert 6 into the following AVL tree and show the rebalancing procedure step by step.



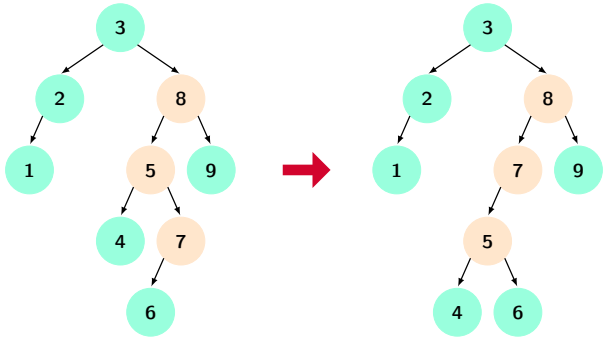
Solution

We insert 6 using the ordinary BST insertion procedure and we obtain the following tree:

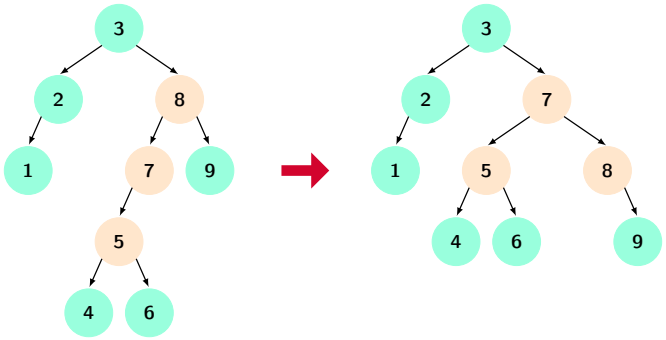


This tree is imbalanced at node 8 since it has a left subtree of height 3 and a right subtree of height 1, leading to a balance factor of 2. Note that the root node 3 is also imbalanced, but we always perform rotations on the lower levels first since this may fix the imbalances at higher levels.

Since node 8 has a taller left subtree and node 5 has a taller right subtree, this is a left-right imbalance so we must perform a double rotation on node 7 to correct it. We perform the first rotation to obtain:



We then perform the second rotation and are left with:



This is a balanced tree, so we are done.

Problem 2. Recall the algorithm for deleting an element from a binary search tree. If that element has no children, we do nothing. If it has one child, we can simply remove it and move its child upwards. Otherwise, if the node has two children, we replace the node we are deleting with its *successor* (and if the successor has one child, move the successor child upwards to be a child of the previous parent of the successor). This algorithm works because of the following fact: the successor of a node with two children has no left child. Prove this fact.

(a) First prove that the successor of a node x with two children must be contained in the right subtree of x.

(b) Use this fact to prove that the successor of x cannot have a left child.

Solution

This problem is related to the correctness of the deletion operation in the binary search trees.

(a) Lets denote the successor of x by s. Recall that the successor of a node is the minimum value node that is greater than x. Suppose for contradiction purposes that x and s have a common ancestor $a \neq x, s$ such that x and s are in different subtrees of a. Then as $x < s$, it must be the case that x is in the left subtree of a and s is in the right subtree of a. But this would imply that $x < a < s$ which contradicts the fact that s is the successor of x. Therefore it must be the case that either x is an ancestor of s, or s is an ancestor of x.

Suppose for contradiction purposes that s is an ancestor of x. Then x must be in the left subtree of s as $x < s$. Now let r denote the right child of x. We have that $x < r$. As r is in the left subtree of s, we have that $r < s$. Combining these two facts we get that $x < r < s$, which contradicts the fact that s is the successor of x. Therefore it must be the case that x is an ancestor of s. And, as $x < s$, s needs to be in right subtree of x.

(b) Suppose for contradiction purposes that s had a left child t. By the BST property we have $t < s$. But as t is contained in the right subtree of x by item (a), we also have that $x < t$. Combining these two facts we get $x < t < s$, which contradicts the fact that s is the successor of x. Therefore the successor s of x cannot have a left child.

Putting the facts together for the purposes of BST correctness analysis: whenever we are deleting a BST node x that has two children and replacing it with its successor s, s will either have no subtrees or only a right subtree (which can then be moved upwards to be a child of the previous parent of node s).

Problem 3. We know that when inserting an element into a binary search tree, there is only one valid place to put that item in the tree (and that is an important property for the correctness of binary search trees). Let's prove this fact rigorously. Let T be a binary search tree and let x be an integer not contained in T. Prove that exactly one of the following statements is true:

- Statement 1: The successor of x has no left child.
- Statement 2: The predecessor of x has no right child.

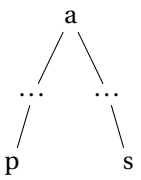
That is, prove that it cannot be the case that both of these statements are true simultaneously, nor that both of these statements are false simultaneously. This implies that there is a unique insertion point for x since upon insertion x must either become the left child of its successor or the right child of its predecessor.

Note: the fact that we are proving (that every element has a unique insertion point) is still true when the element we are inserting would be the minimum or maximum value in the tree, but the two statements we have given do not make sense in this case. Think about why this is true!

Solution

Let the predecessor of x be p, and the successor be s. We know that $p < x < s$, and that there is no key k in the tree such that $p < k < x$ or $x < k < s$.

We claim that either s is an ancestor of p, or p is an ancestor of s. Suppose for contradiction that neither is an ancestor of the other, and therefore p and s have a common ancestor a such that p and s are in different subtrees of a. Notice that in this case, as $p < s$, p would need to be in the left subtree of a and s in the right subtree:



Then we know that $p < a < s$. As x is not in the tree and therefore $x \neq a$, that means that either $p < a < x$ or $x < a < s$. But by the definitions of successor and predecessor both of those cases are impossible. So we have a contradiction, meaning that:

- either s is an ancestor of p;
- or p is an ancestor of s.

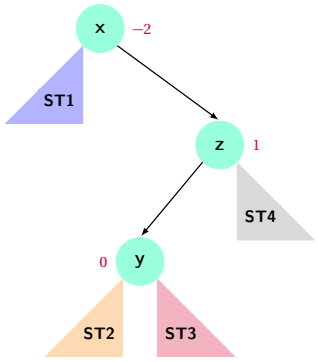
Let's analyse the case in which s is an ancestor of p. In this case, s has a left child as p needs to be in the left subtree of s by the properties of a BST, and thus Statement 1 is false. Assume for contradiction purposes that p has a right child denoted by r. Since r is a right child of p, it holds that $p < r$. Since r is in the left subtree of s, it holds that $r < s$. Therefore $p < r < s$. As x is not in the tree and therefore $x \neq r$, that means that either $p < r < x$ or $x < r < s$. But by the definitions of successor and predecessor both of those cases are impossible. So we have a contradiction and therefore p cannot have a right child, which

implies that Statement 2 is true. Therefore, in the case in which s is an ancestor of p, Statement 1 is false and Statement 2 is true.

Let's analyse the case in which p is an ancestor of s. In this case, p has a right child as s needs to be in the right subtree of p by the properties of a BST, and thus Statement 2 is false. Assume for contradiction purposes that s has a left child denoted by ℓ. Since ℓ is a left child of s, it holds that $\ell < s$. Since ℓ is in the right subtree of p, it holds that $p < \ell$. Therefore $p < \ell < s$. As x is not in the tree and therefore $x \neq \ell$, that means that either $p < \ell < x$ or $x < \ell < s$. But by the definitions of successor and predecessor both of those cases are impossible. So we have a contradiction and therefore s cannot have a left child, which implies that Statement 1 is true. Therefore, in the case in which p is an ancestor of s, Statement 1 is true and Statement 2 is false.

Notice that by following the insertion procedures of a BST, x will be inserted as a right child of p in the case in which s is an ancestor of p, otherwise it will be inserted as a left child of s.

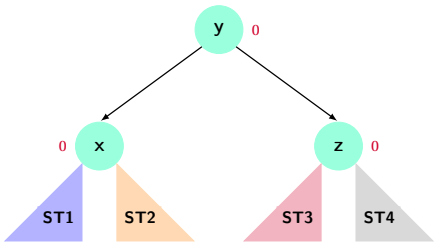
Problem 4. Consider the binary search tree shown below, with balance factors as indicated. Assume that the subtrees **ST1**, **ST2**, **ST3** and **ST4** are AVL trees. Show that performing the rotation algorithm for the right-left case results in an AVL tree.



Solution

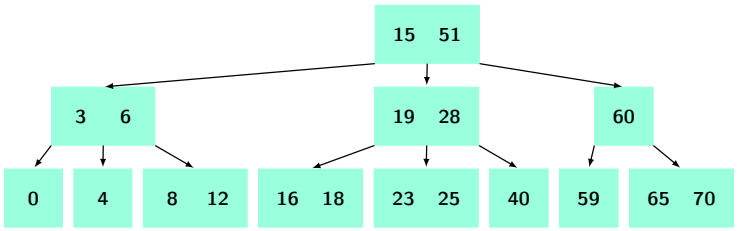
Let height(**ST2**) be h. We can deduce that height(**ST3**), height(**ST4**) and height(**ST1**) are also h (based on the balance factors).

After applying the two phases of the right-left case rotations, we obtain the following situation:



Based on the heights of **ST1**, **ST2**, **ST3** and **ST4** we can deduce the balance factors of x, y and z as all being 0. Since **ST1**, **ST2**, **ST3** and **ST4** are AVL trees, the balance factors of all nodes in the tree are in $\{-1, 0, 1\}$. So the tree is now an AVL tree.

Problem 5. Consider the following 2-3 Search Tree:



(a) Visualise the state of the tree after sequentially performing each of the following actions:

i. Insert key 5

ii. Insert key 62

iii. Delete key 59

iv. Insert key 10

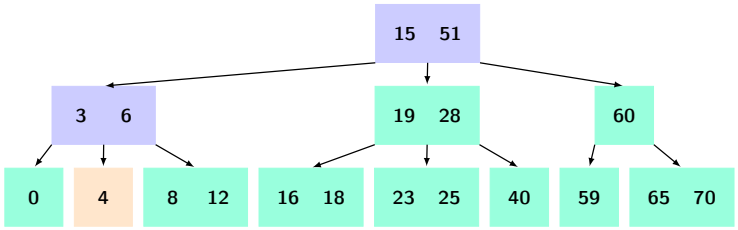
v. Delete key 40

Note: We are only covering deletion if the key is a leaf node.

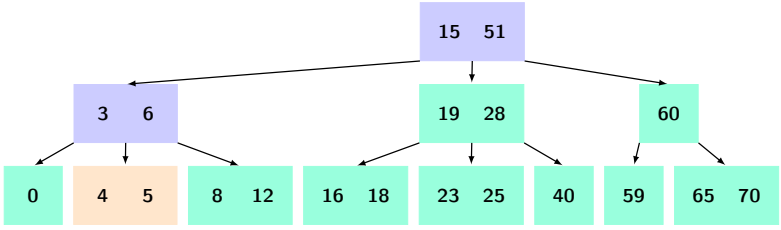
(b) Convert the resulting 2-3 Search Tree from part (a) into the equivalent Left-Leaning Red-Black Tree.

Solution

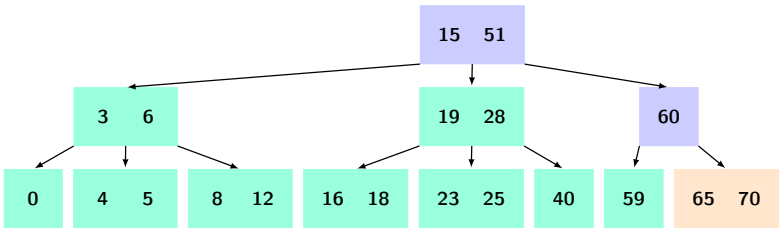
(a) i. To insert key 5, we must first identify if it exists or not. This traversal of the tree will indicate where we can place key 5 if it is not present in the tree.



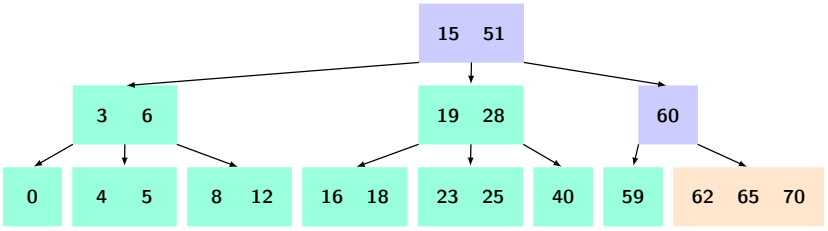
The orange-highlighted 2-Node is where key 5 can be inserted. Since it is a 2-Node, we can simply just add the new key by converting the leaf into a 3-Node.



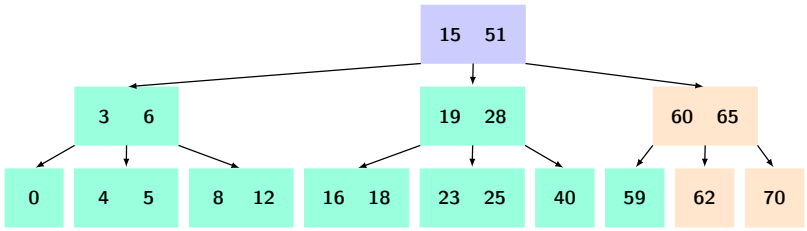
ii. To insert key 62, we traverse down the tree to find it's corresponding position. This path has been highlighted in the tree below.



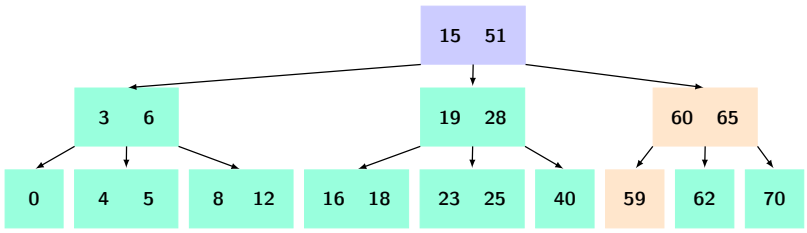
We insert 62 into the 3-Node leaf, creating a 4-Node.



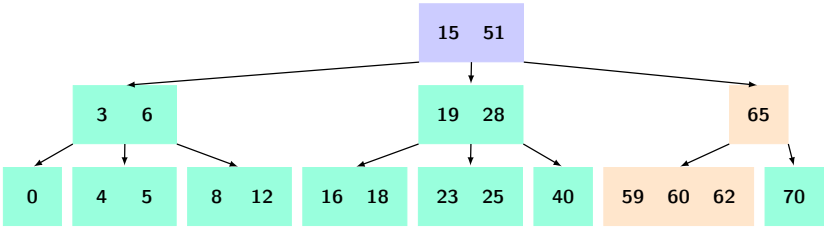
A 4-Node is not allowed to exist and we must remedy this by promoting the middle element of the 4-Node into the above level, whilst leaving the two remaining keys as individual 2-Nodes. In doing so every node now satisfies the constraints.



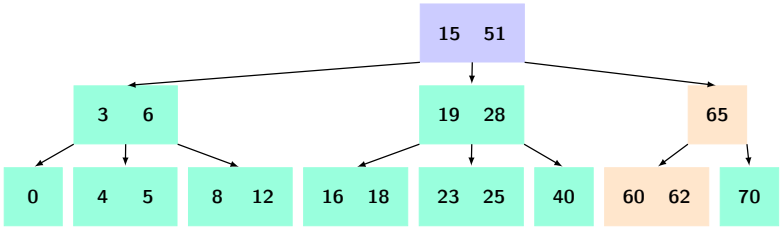
iii. To delete key 59, we must make sure to always maintain the invariant that the node we are currently processing is larger than a 2-Node. We start the traversal at the root, making sure it meets the criteria imposed by the invariant, which it does. We continue to traverse and reach the 3-Node containing keys 60 and 65.



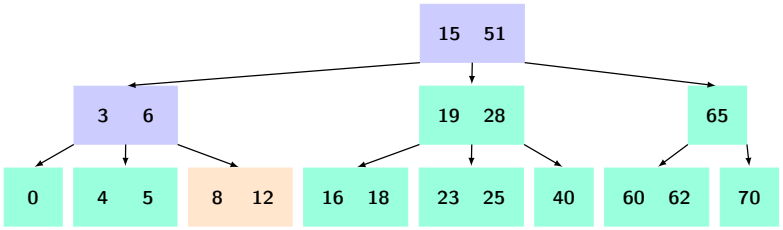
Our next target cell to reach is the 2-Node containing key 59. However, we must maintain the invariant and therefore must do some adjustments. The immediate neighbour of 59 is a 2-Node and hence we can combine the two 2-Nodes and demote the current cell in the 3-Node we are in (key 60) into a 4-Node.



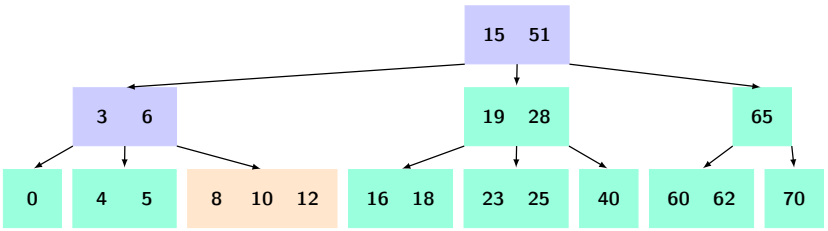
Finally, we can recurse into the leaf 4-Node. Since we are at a leaf and it is larger than a 2-Node, we can safely delete the target key.



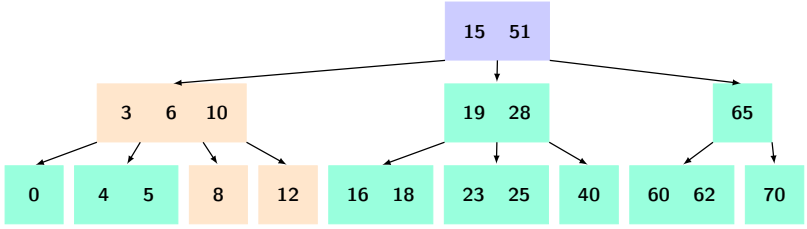
iv. A simple traversal down the tree can determine where the key 10 should be inserted.



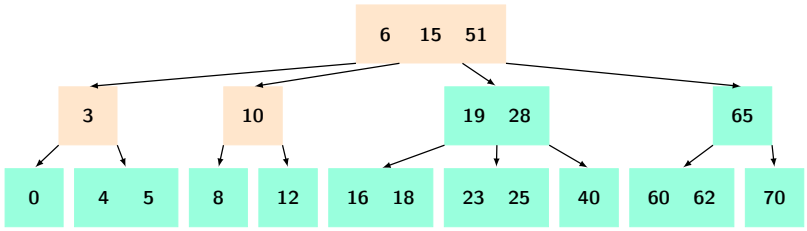
The highlighted path shows that the leaf is a 3-Node, hence an insertion will cause it to become a 4-Node.



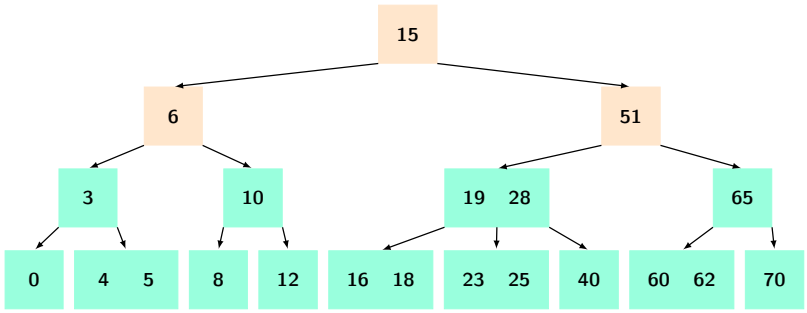
To deal with this situation, we perform the insertion and then promote the middle element of the 4-Node into the above level. The remaining two nodes will become individual 2-Nodes.



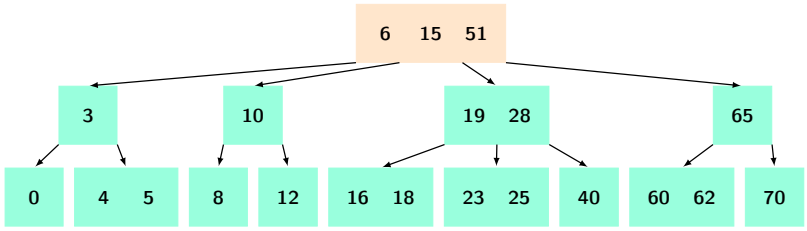
We must make sure to recursively perform this adjustment until we are at the root.



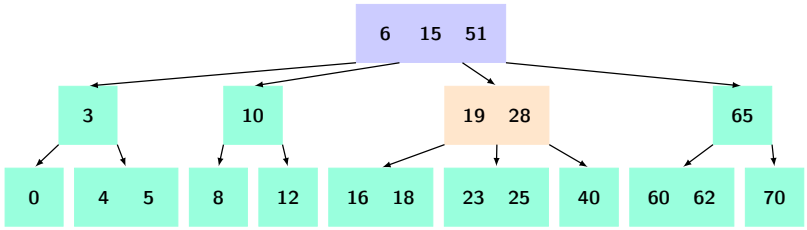
If the root also becomes a 4-Node, as is this case, we promote the middle element above as the new root. Note that this is the only way a 2-3 Search Tree gains height.



v. Deleting key 40 will require to consider a series of unique cases in order to preserve the special rules of the 2-3 Search Tree. As in part iii. we must always process the node we intend to go to so that it is larger than a 2-Node. This processing also applies to the root before we start the recursion.

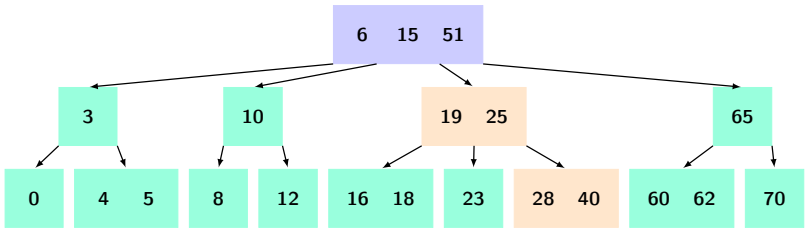


Now that the root has been processed, we can begin at the root. The next node that allows us to reach key 40 is the 3-Node holding keys 19 and 28. Since this is already larger than a 2-Node, we can safely recurse into it.

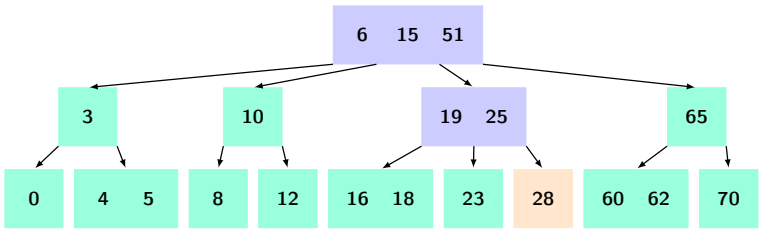


Next, we can directly access key 40. However, since this is a 2-Node we must make some ad-

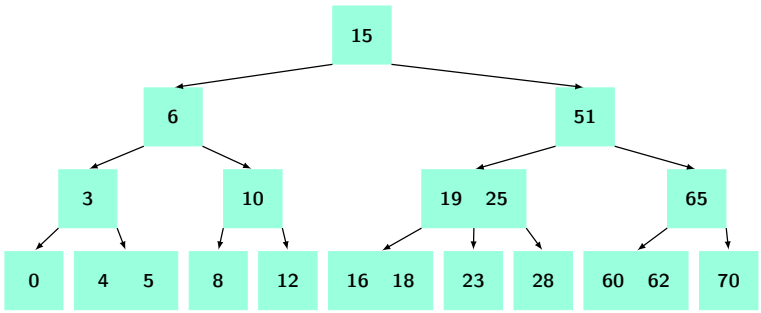
justments. The immediate sibling of key 40 is not another 2-Node. Because the immediate sibling is a 3-Node, we can rotate an element from it into the current node, which rotates another element into the 2-Node containing key 40. Specifically, we will move the largest key of the sibling into the current, and the largest key of the current into the target 2-Node.



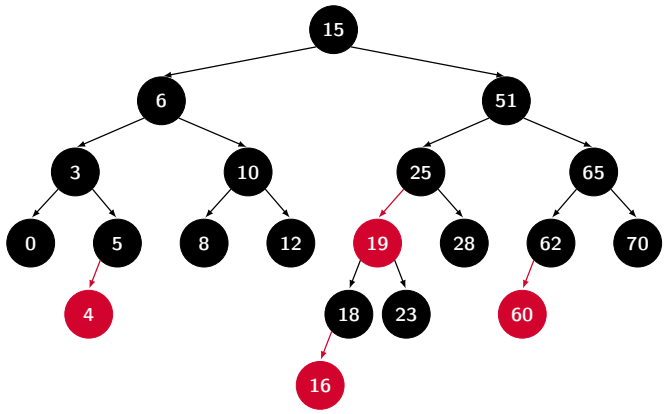
We can now recurse into the final leaf node and safely delete key 40.



However, we must always make sure we fix any remaining 4-Nodes that may have formed as we exit the recursion. In this case, the root node remains a 4-Node and will be adjusted by promoting the middle element, key 15, to a higher level.



(b) The 2-3 Search Tree can be converted to the Left Leaning Red-Black Tree shown below:



Problem 6. Sequentially perform the following operations on an empty Left-Leaning Red-Black-Tree. For each operation; (1) draw the resulting Red-Black-Tree, (2) the equivalent 2-3 Search Tree, and (3) state the black height of the Red-Black-Tree

(a) Insert key 40

(b) Insert key 20

(c) Insert key 5

(d) Insert key 1

(e) Insert key 32

(f) Insert key 64

(g) Insert key 13

(h) Insert key 5

(i) Insert key 38

(j) Insert key 36

Solution

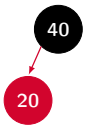
When performing insertions into a RBT all nodes are inserted as red nodes. We then might need to a sequence of left rotations, right rotations, or colour flips to maintain the properties of the tree. We keep recursively performing these back up to the root until all issues are fixed.

(a) Inserting 40 we have a single red node.

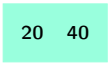
When the root turns red we always turn it back to black.

The black height is 0 and below is the equivalent 2-3 Search Tree.

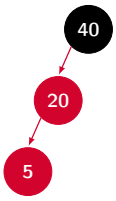
(b) Inserting 20 it becomes the left child of 40. No other action is needed.



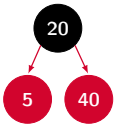
The black height is 0 and below is the equivalent 2-3 Search Tree.



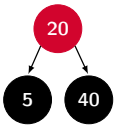
(c) Inserting 5 it becomes the left child of 20.



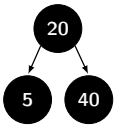
Examining 40 we see it has a red left child 20, and 20's left child is also red. We perform a right rotation.



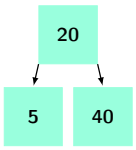
Examining 20, we see that is has 2 red children, and thus the colours will be flipped.



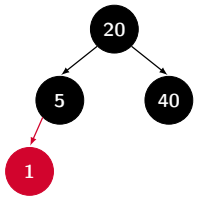
The root will be turned back to black.



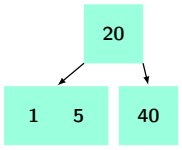
The black height is 1 and below is the equivalent 2-3 Search Tree.



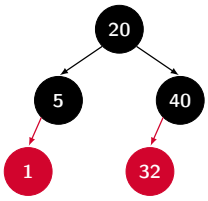
(d) Inserting 1 it becomes the left child of 5. No other action is needed.



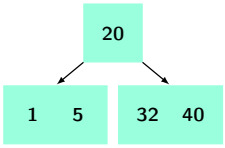
The black height is 1 and below is the equivalent 2-3 Search Tree.



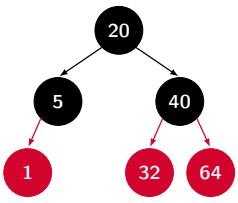
(e) Inserting 32 it becomes the left child of 40. No other action is needed.



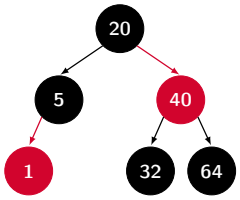
The black height is 1 and below is the equivalent 2-3 Search Tree.



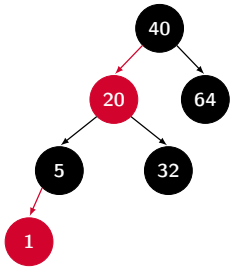
(f) Inserting 64 it becomes the right child of 40.



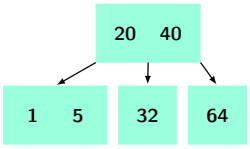
Since 40 now has 2 red children the colours flip and 40 becomes red now.



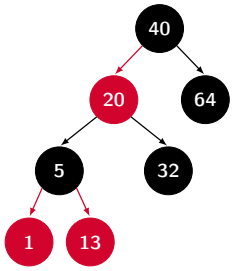
Now examining 20 we see that its only red child is a right child, we perform a left rotation.



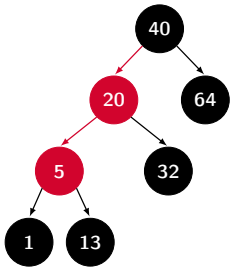
The black height is 1 and below is the equivalent 2-3 Search Tree.



(g) Inserting 13 it becomes the right child of 5.

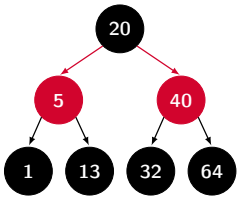


Since 5 now has 2 red children the colours flip and 5 becomes red now.

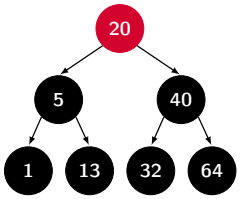


Recursing up and examining 40 we see it has a red left child 20, and 20's left child is also red. We

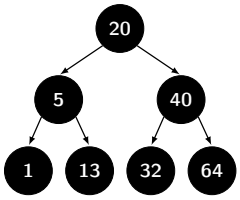
perform a right rotation.



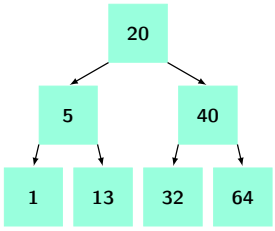
Examining 20 now we see that it has 2 red children so the colours flip.



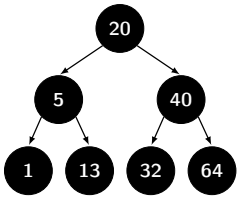
Since 20 is the root it will turn to black.



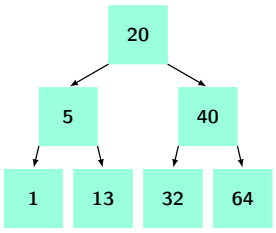
The black height is 2 and below is the equivalent 2-3 Search Tree.



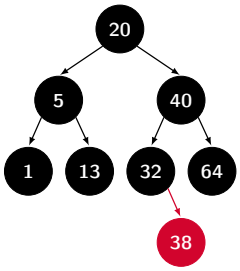
(h) Inserting 5, following the insertion path we notice the key already exists. The tree is unaltered.



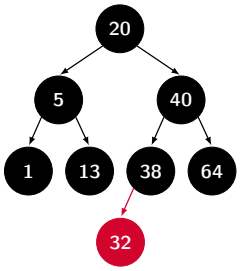
The black height is 2 and below is the equivalent 2-3 Search Tree.



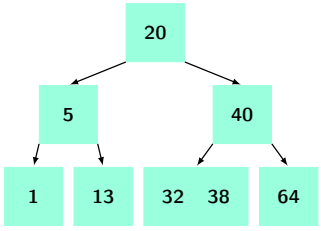
(i) Inserting 38 we see it becomes the right child of 32.



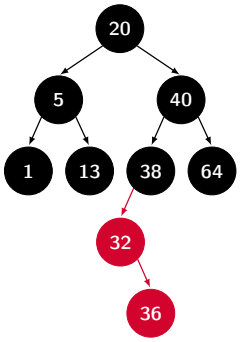
Since 32's only red child is a right child, we perform a left rotation.



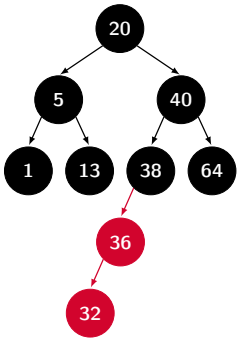
The black height is 2 and below is the equivalent 2-3 Search Tree.



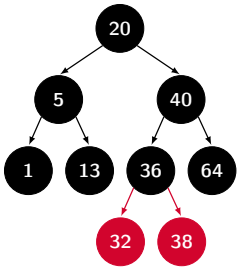
(j) Inserting 36 it becomes the right child of 32.



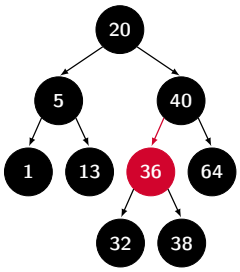
Since 32 only red child is a right child, we perform a left rotation.



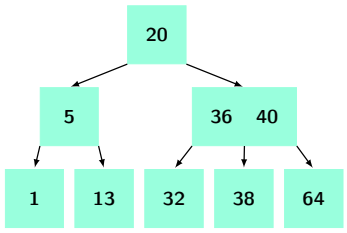
Examining 38 we see that we see it has a red left child 36, and 36's left child is also red. We perform a right rotation.



Since 36 has 2 red children the colour flip and we pass the red upwards.



The black height is 2 and below is the equivalent 2-3 Search Tree.



The overall black height of the tree increases only when a red node is propagated up to the root, temporarily making the root red. However, since we immediately change the root back to black (without further modifying the colours of other nodes), the black height increases by 1. Keep in mind that the black height corresponds to the height of the equivalent 2-3 Search Tree. The colour changes in the Red-Black Tree mirror the process of passing up nodes in a 2-3 Search Tree. Eventually, the root of the 2-3 Search Tree temporarily becomes a 4-node, which then splits and promotes a node, increasing the tree's height by one level.

Problem 7. You are given a set of distinct keys $x_1, x_2, ..., x_n$.

(a) Design an algorithm that creates a binary search tree of minimal height containing those keys in $O(n\log(n))$ time.

(b) Prove that your algorithm produces a BST of height at most $\log(n)$.

(c) Prove that the fastest algorithm for this problem in the comparison-based model has time complexity $\Theta(n\log(n))$.

Solution

(a) First we sort the keys, this takes $O(n\log(n))$. We take the median element of the sequence and insert it into a BST, then recursively do the same thing with the resulting left and right sublists. In the below code, we assume we have a *Node* class which contains a key, a left child, and a right child. We must sort the list before calling OPTIMAL_BST(x[1..n]).

```
1: function OPTIMAL_BST(x[1..n])
2:     if x is empty then return null
3:     mid = ⌊n/2⌋
4:     root = Node(x[mid])
5:     root.left = OPTIMAL_BST(x[1..mid-1])
6:     root.right = OPTIMAL_BST(x[mid+1..n])
7:     return root
```

(b) To prove that the height of the constructed tree is at most $\log(n)$, we write a recurrence for the height.

$$H(n) = 1 + H\left(\left\lceil \frac{n-1}{2} \right\rceil\right) \leq 1 + H\left(\frac{n}{2}\right)$$

Using telescoping,

$$\begin{aligned} H(n) &\leq 1 + \left[1 + H\left(\frac{n}{4}\right)\right], \\ &\leq 2 + \left[1 + H\left(\frac{n}{8}\right)\right], \\ &\leq ..., \\ &\leq k + H\left(\frac{n}{2^k}\right). \end{aligned}$$

We know that $H(1) = 0$, so setting $k = \log(n)$, we obtain

$$H(n) \leq \log(n) + H(1) = \log(n)$$

as required.

(c) If we could construct any BST from n keys in faster than $O(n\log(n))$ time, we could then do an in-order traversal of the resulting BST to obtain the keys in sorted order. This means we could sort faster than $O(n\log(n))$, but $\Omega(n\log(n))$ is a lower bound for comparison-based sorting, so such an algorithm is impossible.

Supplementary Problems

Problem 8. In the lecture recordings we claimed that AVL trees are good because the balance property guarantees that the tree always has height $O(\log(n))$. Let's prove this.

(a) Write a recurrence relation for $n(h)$, the **minimum** number of nodes in an AVL tree of height h. [Hint: It should be related to the Fibonacci numbers]

(b) Find an exact solution to this recurrence relationship in terms of the Fibonacci numbers[1]. [Hint: Compare the sequence with the Fibonacci sequence, find a pattern, and then prove that your pattern is right using induction]

(c) Prove using induction that the Fibonacci numbers satisfy $F(h) \geq 1.5^{h-1}$ for all $h \geq 0$

(d) Using (a), (b), and (c), prove that a valid AVL tree with n elements has height at most $O(\log(n))$

Solution

Consider an AVL tree of height h whose root node has the two subtrees L and R. Suppose that L and R have the same height. We could remove the nodes from the lowest level of one of the two subtrees, making them differ in height by one and it would still be a valid AVL tree with the same height. Therefore an AVL tree with the fewest possible nodes must have L and R differing in height by one, hence they must have heights $h-1$ and $h-2$. Lastly, note that L and R must also be AVL trees with the fewest possible nodes. Therefore the minimum number of nodes in an AVL tree satisfies the following recurrence relationship:

$$n(h) = \begin{cases} 1 & \text{if } h = 0, \\ 2 & \text{if } h = 1, \\ 1 + n(h-1) + n(h-2) & \text{if } h > 0. \end{cases}$$

This looks suspiciously similar to the Fibonacci sequence, so lets compare them:

h	0	1	2	3	4	5	6	7	8
$F(h)$	1	1	2	3	5	8	13	21	34
$n(h)$	1	2	4	7	12	20	33	54	88

This table strongly suggests the pattern

$$n(h) = F(h+2) - 1$$

Let's prove this by induction. We have $n(0) = F(2) - 1 = 1$ and $n(1) = F(3) - 1 = 2$ as required. Now suppose $n(h) = F(h+2) - 1$ for all $h \leq k$ for some value of k. We need to show that $n(k+1) = F(k+3) - 1$. From the recurrence, we have

$$n(k+1) = 1 + n(k) + n(k-1).$$

[1] For this problem, index the Fibonacci numbers from zero with the base case $F(0) = F(1) = 1$.

Invoking the inductive hypothesis, we can write

$$\begin{aligned} n(k+1) &= 1 + F(k+2) - 1 + F(k+1) - 1, \\ &= F(k+2) + F(k+1) - 1 \end{aligned}$$

By the definition of Fibonacci numbers, $F(k+2) + F(k+1) = F(k+3)$, hence we have

$$\begin{aligned} n(k+1) &= F(k+2) + F(k+1) - 1, \\ &= F(k+3) - 1, \end{aligned}$$

as required. Hence by strong induction on k, we have $n(h) = F(h+2) - 1$ for all h.

Next, we prove that the Fibonacci numbers have the desired bound. In the base case, we have

$$F(0) = 1 \geq 1.5^{-1}, \qquad F(1) = 1 \geq 1.5^{0}.$$

Now, suppose that $F(h-1) \geq 1.5^{h-2}$ and $F(h) \geq 1.5^{h-1}$. We need to prove that $F(h+1) \geq 1.5^{h}$. From the definition of Fibonacci numbers, we have

$$F(h+1) = F(h) + F(h-1),$$

using the inductive hypothesis, we can write the bound

$$\begin{aligned} F(h+1) &\geq 1.5^{h-1} + 1.5^{h-2}, \\ &= 1.5 \times 1.5^{h-2} + 1.5^{h-2}, \\ &= (1.5+1) \times 1.5^{h-2}, \\ &\geq 2.25 \times 1.5^{h-2}, \\ &= (1.5)^2 \times 1.5^{h-2}, \\ &= 1.5^{h}. \end{aligned}$$

Therefore by induction on h, we have $F(h) \geq 1.5^{h-1}$ for all h.

Finally, combining the above, we have $n(h) = F(h+2) - 1 \geq 1.5^{h+1} - 1$. Rearranging, we find $1.5^{h+1} \leq n(h) + 1$, and hence

$$h + 1 \leq \log_{1.5}(n(h)+1) \qquad \Longrightarrow \qquad h = O(\log(n)),$$

as required.